

# DeepPersona 코드 학습교재

모듈 05 — 깊이우선 값 채우기: `generate_profile.py` (Step 5)

---

학습 정리 데크 (Claude와 함께)

September 13, 2026

원저: Wang et al., *DeepPersona* (arXiv:2511.07338) · 논문 §3.2 "Progressive LLM filling"

# 이 모듈의 목표

**한눈에** 고른 슬롯에 LLM이 **값**을 채운다 — 카테고리를 순서대로, 앞의 결과를 맥락으로 받아 일관되게.

## 얻는 것

- **depth-first** 순차 생성의 뜻
- 카테고리별 **조건부 프롬프트**
- 값 생성 코드(1 카테고리=1 LLM 호출)
- **Summary** 서사 생성
- **카테고리 drop 버그**(19→6)
- ▶ 이 모듈이 식 (2)의  $generator Pr_{\theta}(v_i | a_i, S, P_{<i})$ 에 해당.

## 핵심 통찰

- 각 카테고리는 앞의 모든 카테고리를 reference로 받음 → 전역 일관성
- 값은 **≤100자**로 캡 → 얇은 narrative
- 6개만 명시 처리 → **13개 조용히 누락**

1. Progressive filling
2. 카테고리 순차 생성
3. 카테고리 drop 버그

# 1. Progressive filling

---

## 깊이우선 — 앞을 보고 다음을 쓴다

**쉽게 말하면** 이력서를 위에서부터 채우듯, 인구통계 → 직업 → 가치관 순으로, 매번 지금까지 쓴 것을 보며 다음 칸을 채운다.

### 논문 "Progressive LLM filling"

- selector가 슬롯  $a_i$ 를 고르면, generator  $\theta$ 가 값  $v_i$ 를 채움
- $v_i$ 는 지금까지의 프로필  $P_{<i}$ 에 **조건부**
- 그래서 뒤 값이 앞 값과 안 어긋남(전역 일관성)

#### 독립 생성이면?

각 슬롯을 따로 채우면 "35세 은퇴 CEO인데 대학생" 같은 모순 발생.

#### 조건부 생성이면

직업을 쓸 때 이미 만든 나이·인구통계를 보고 씀  
→ 앞뒤 일치.

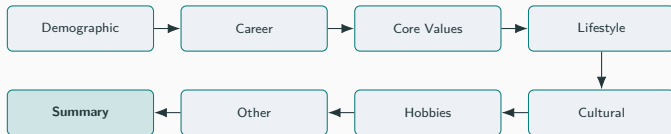
**한 눈에** 진입점 `generate_single_profile(attribute_count=200)` — 시드→선택→값 채우기→Summary를 한 번에.

## 2. 카테고리 순차 생성

---

## 7개 카테고리를 **순서대로**, 앞을 참조하며

**쉽게 말하면** 카테고리를 하나씩 채우되, 각 단계 프롬프트에 앞서 만든 카테고리를 통째로 "참고자료"로 넣는다.



- 각 화살표 = "앞 카테고리 JSON을 for reference 로 프롬프트에 주입".
- 예: Career 프롬프트엔 이미 만든 Demographic이, Hobbies엔 Demo+Career+Values+Lifestyle+Cultural 전부가 들어감.

**한눈에** 이 "누적 참조"가 곧  $P_{<i}$  조건부 — 코드로 구현된 **depth-first breadth**.

## 조건부 프롬프트 — 앞 결과를 통째로 넣는다

```
career_input = (  
    "Base Information (for reference):\n" + json.dumps(base_info,...) + "\n\n"  
    "Life Story (for reference):\n" + str(life_story) + "\n\n"  
    "Demographic Information (for reference):\n"           # <- 앞서만든것 !  
    + json.dumps(profile.get("Demographic Information", {}),...) + "\n\n"  
    "Instructions: ... develop the 'Career and Work Identity' ... "  
    "evident or reasonably inferred from base_info, life_story, Demographic ...")
```

- **누적**: 뒤 카테고리일수록 `profile.get("...")` 로 앞 카테고리를 더 많이 참조.
- **"inferred from"**: 새 사실을 지어내지 말고 앞 정보에서 추론하라고 지시 → 일관성.
- **생성 순서** = `Demographic → Career → Core Values → Lifestyle → Cultural → Hobbies → Other`.

▶ 프롬프트마다 "balance" 류 단어 금지 등 스타일 제약도 포함(반복 표현 억제).

## 값 생성 — 카테고리 1개 = LLM 호출 1번

```
def collect_leaf_paths(obj, current_path):          # 카테고리아래모든 leaf 수집
    for key, value in obj.items():
        path = f"{current_path}.{key}" if current_path else key
        if isinstance(value, dict) and not value: leaf_paths.append(path) # 빈{}=leaf
        elif isinstance(value, dict): collect_leaf_paths(value, path)
system_prompt = "Format as JSON: key=attribute path, value=... (<=100 characters)."
user_prompt = custom_prompt + "\n".join(f"- {p}" for p in leaf_paths)
response = get_completion(messages)                # 한번에전체 leaf 값생성
generated_values = json.loads(cleaned_response)     # {path: value, ...}
```

- leaf 수집: 카테고리 하위의 빈 {} (=슬롯)을 경로로 모음.
- 1회 호출: 그 카테고리의 모든 슬롯을 한 번의 LLM 호출로 채움(호출 수 절약).
- 값 캡 <=100자: 각 값이 짧게 강제됨 → narrative가 알아지는 원인.
- 파싱: 마크다운 ```json 펜스를 벗기고 json.loads → {경로:값}.

▶ 이후 parts=path.split('.') 로 {경로:값}을 다시 중첩 dict로 복원해 프로필에 넣음.

## Summary — 전체를 **서사**로 엮는다

```
profile_for_summary = profile.copy()
for key in ['base_info', 'personal_story', 'interests', 'Occupations']:
    profile_for_summary.pop(key, None)          # 원재료제거중복 ( 방지)
final_summary_text = generate_final_summary(profile_for_summary, base_info)
profile["Summary"] = final_summary_text        # 100~400 단어인칭 1 서사
```

- **입력**: 완성된 구조적 프로필 전체(원재료 키는 제거).
- **출력**: 100~400단어의 통합 narrative(부족하면 경고, 초과하면 400단어로 절삭).
- **역할**: 논문의  $\text{Narr}(P)$  — 구조적 속성을 사람이 읽는 이야기로. (Fig 2 우하단 "Summary")

**한눈에** 산출물 = 구조적 속성(200개) + 서사 Summary. 이게 한 페르소나  $P$ .

### 3. 카테고리 drop 버그

---

## 버그: 6개만 처리, 13개 조용히 누락

```
demographic_template = selected_paths.get("Demographic Information")
career_template      = selected_paths.get("Career and Work Identity")
core_template        = selected_paths.get("Core Values, Beliefs, and Philosophy")
lifestyle_template   = selected_paths.get("Lifestyle and Daily Routine")
cultural_template    = selected_paths.get("Cultural and Social Context")
hobbies_template     = selected_paths.get("Hobbies, Interests, and Lifestyle")
other_template       = selected_paths.get("Other Attributes")    # 데이터에 없음 -> None!
```

- **원인 1:** 모듈 02의 top-level이 19개(중복 variant). 그런데 여기서 정확한 6개 문자열만 `.get()`.
- **원인 2:** 나머지 13개(예: "Core Values and Beliefs", "Lifestyle and Habits", "Education and Learning" ...)를 잡을 "Other Attributes" 키가 `selected_paths`에 없음 → `None` → skip.
- **결과:** 19개 중 6개(32%)만 값이 채워지고 13개는 흔적 없이 사라짐.

▶ `selected_paths`는 카테고리별 dict인데, 중복 variant를 "Other Attributes"로 묶는 로직이 없다.

# 버그의 영향 & 수정 방향

**쉽게 말하면** 그래서 광고된 "1MB narrative"가 실제로 27KB에 그친다 — 슬롯은 골라졌는데 값 채우기에서 대부분 버려져서다.

## 실측(count=350, study.md §13)

|         | 실제    | README 주장 |
|---------|-------|-----------|
| 프로필 크기  | 27 KB | ~1 MB     |
| 채워진 속성  | 200   | 200+      |
| 처리 카테고리 | 6/19  | (전부 가정)   |

## 수정 방향

- selected\_paths의 모든 top-level 키를 동적 순회(하드코딩 6개 대신)
- 모듈 02의 merge\_tree.py 로 중복 variant를 12개로 정규화
- 값 캡 100자 완화 → narrative 길이↑

**한눈에** 다음 bottleneck은 임베딩이 아니라 여기(카테고리 매핑)다 — 재현 시 반드시 인지.

## 꼭 기억할 것

- depth-first = 앞 카테고리를 참조하며 채움
- 카테고리 1개 = LLM 호출 1번
- 값  $\leq 100$ 자, Summary는 100~400단어
- 6/19만 처리(카테고리 drop 버그)
- 병목은 임베딩이 아니라 매핑

## 함수 지도

- `generate_single_profile` (진입점)
- `generate_category_attributes` (값 생성)
- `collect_leaf_paths` (슬롯 수집)
- `generate_final_summary` (서사)

**한눈에** 다음 `06_run_reproduce` : 실제로 돌려 페르소나를 재현하고 pkl을 고친다.

## 모듈 05 끝

다음: 06\_run\_reproduce — 실행·재현·스크립트